

Databases I - CPIT240

LAB-4

SQL Data Manipulation Language

(INSERT, UPDATE, DELETE)

EMPLOYEES

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	06/17/1987	AD PRES	24000	-	-	90
101	Neena	Kochhar	NKOCHHAR	515.123.4568	09/21/1989	AD VP	17000	-	100	90
102	Lex	De Haan	LDEHAAN	515.123.4569	01/13/1993	AD VP	17000	-	100	90
103	Alexander	Hunold	AHUNOLD	590.423.4567	01/03/1990	IT_PROG	9000	-	102	60
104	Bruce	Ernst	BERNST	590.423.4568	05/21/1991	IT_PROG	6000	-	103	60
107	Diana	Lorentz	DLORENTZ	590.423.5567	02/07/1999	IT_PROG	4200	-	103	60
124	Kevin	Mourgos	KMOURGOS	650.123.5234	11/16/1999	ST_MAN	5800	-	100	50
141	Trenna	Rajs	TRAJS	650.121.8009	10/17/1995	ST_CLERK	3500	-	124	50
142	Curtis	Davies	CDAVIES	650.121.2994	01/29/1997	ST_CLERK	3100	-	124	50
143	Randall	Matos	RMATOS	650.121.2874	03/15/1998	ST_CLERK	2600	-	124	50
144	Peter	Vargas	PVARGAS	650.121.2004	07/09/1998	ST_CLERK	2500	-	124	50
149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	01/29/2000	SA_MAN	10500	.2	100	80
174	Ellen	Abel	EABEL	011.44.1644.429267	05/11/1996	SA_REP	11000	.3	149	80
176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	03/24/1998	SA_REP	8600	.2	149	80
178	Kimberely	Grant	KGRANT	011.44.1644.429263	05/24/1999	SA_REP	7000	.15	149	-
										row(s) 1 - 15 of 20 >

DEPARTMENTS

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
10	Administration	200	1700
20	Marketing	201	1800
50	Shipping	124	1500
60	IT	103	1400
80	Sales	149	2500
90	Executive	100	1700
110	Accounting	205	1700
190	Contracting	-	1700
			row(s) 1 - 8 of 8

JOB_GRADES

GRADE_LEVEL	LOWEST_SAL	HIGHEST_SAL
A	1000	2999
B	3000	5999
C	6000	9999
D	10000	14999
E	15000	24999
F	25000	40000
		row(s) 1 - 6 of 6

Tables Used in this Part

EMPLOYEES

Column Name	Data Type	Nullable	Default	Primary Key
EMPLOYEE_ID	NUMBER(6,0)	No	-	1
FIRST_NAME	VARCHAR2(20)	Yes	-	-
LAST_NAME	VARCHAR2(25)	No	-	-
EMAIL	VARCHAR2(25)	No	-	-
PHONE_NUMBER	VARCHAR2(20)	Yes	-	-
HIRE_DATE	DATE	No	-	-
JOB_ID	VARCHAR2(10)	No	-	-
SALARY	NUMBER(8,2)	Yes	-	-
COMMISSION_PCT	NUMBER(2,2)	Yes	-	-
MANAGER_ID	NUMBER(6,0)	Yes	-	-
DEPARTMENT_ID	NUMBER(4,0)	Yes	-	-
				1 - 11

DEPARTMENTS

Column Name	Data Type	Nullable	Default	Primary Key
DEPARTMENT_ID	NUMBER(4,0)	No	-	1
DEPARTMENT_NAME	VARCHAR2(30)	No	-	-
MANAGER_ID	NUMBER(6,0)	Yes	-	-
LOCATION_ID	NUMBER(4,0)	Yes	-	-
				1 - 4

JOB_GRADES

Column Name	Data Type	Nullable	Default	Primary Key
GRADE_LEVEL	VARCHAR2(3)	Yes	-	-
LOWEST_SAL	NUMBER	Yes	-	-
HIGHEST_SAL	NUMBER	Yes	-	-
				1 - 3

SQL Statements Types

- Data Definition Language (**DDL**) Statements.
- Data Manipulation Language (**DML**) Statements.
- Transaction Control Language (**TCL**) Statements.

DDL	DML	TCL
CREATE	SELECT	COMMIT
ALTER	INSERT	ROLLBACK
DROP	UPDATE	SAVEPOINT
TRUNCATE	DELETE	
RENAME	MERGE	
GRANT		
REVOKE		
COMMENT		

Data Manipulation Language

- Data manipulation language (DML) statements **access** and **manipulate** data in existing tables.

INSERT

- You can **add new rows** to a table by issuing the **INSERT** statement.

- **SQL STATEMENT**

INSERT INTO **table** [(**column** [, **column**...])]

VALUES (**value** [, **value**...]);

- **In the syntax:**

- **table** is the name of the table
- **column** is the name of the column in the table to populate
- **value** is the corresponding value for the column

- **NOTE:** This statement with the VALUES clause adds **only one row** at a time to a table.

TRY THIS

TRY THIS

1. Type the following:

```
INSERT INTO departments(department_id,  
                        department_name, manager_id, location_id)  
VALUES (70, 'Public Relations', 100, 1700);
```

2. You should see:

```
1 row created.
```

INSERT

- Because you can insert a new row that contains values for each column, **the column list is not required** in the INSERT clause.
- However, if you do not use the column list, **the values must be listed according to the default order** of the columns in the table, and a value must be provided for each column.
- To find out what **the default order** is, use the DESCRIBE command.

INSERT

- Enclose character and date values in **single quotation** marks.
- It is **not** recommended that you enclose **numeric values** in single quotation marks.

INSERTING ROWS WITH NULL VALUES

1. **Implicit method:** Omit the column from the column list.

TRY THIS

1. Type the following:

```
INSERT INTO departments (department_id, department_name)
VALUES      (30, 'Purchasing');
```
2. You should see:

```
1 row created.
```

2. **Explicit method:** Specify the NULL keyword in the VALUES clause.

TRY THIS

1. Type the following to see the order of the columns in the table:

```
DESCRIBE departments;
```
2. Type the following to insert a new row:

```
INSERT INTO      departments
VALUES      (100, 'Finance', NULL, NULL);
```
3. You should see:

```
1 row created.
```

INSERT

- You can use **functions** to enter special values in your table.

TRY THIS

1. Type the following:

```
INSERT INTO employees (employee_id,  
                        first_name, last_name,  
                        email, phone_number,  
                        hire_date, job_id, salary,  
                        commission_pct, manager_id,  
                        department_id)  
  
VALUES (113,  
        'Louis', 'Popp',  
        'LPOPP', '515.124.4567',  
        SYSDATE, 'AC_ACCOUNT', 6900,  
        NULL, 205, 100);
```

2. You should see:

1 row created.

INSERT

- You can save commands with **substitution variables** to a file and execute the commands in the file.
- A substitution variable is **preceded by** one or two ampersands (&).
- When SQL*Plus encounters an **undefined substitution variable** in a command, SQL*Plus **prompts you for the value**.

TRY THIS

TRY THIS

1. Open Notepad and save the following command in a file. Call it

`insert_department.sql:`

`INSERT INTO departments`

`(department_id, department_name, location_id)`

`VALUES (&department_id, '&department_name', &location);`

2. Use the **START** command to run the script:

`START C:\...\insert_department.sql`

3. Insert the following when prompted:

`Enter value for department_id: 40`

`Enter value for department_name: Human Resources`

`Enter value for location: 2500`

4. You should see:

`1 row created.`

COPYING ROWS FROM ANOTHER TABLE

- You can use the INSERT statement to add rows to a table where the values are derived from existing tables.
- In place of the VALUES clause, you use a subquery.

- SQL STATEMENT

INSERT INTO table [column (, column)] subquery;

- In the syntax:
 - table is the table name.
 - column is the name of the column in the table to populate.
 - subquery is the subquery that returns rows to the table.
- The number of columns and their data types in the column list of the INSERT clause must match the number of values and their data types in the subquery.

TRY THIS

TRY THIS

1. Type the following:

```
DROP TABLE SALES_REPS;
```

```
CREATE TABLE sales_reps(  
  id NUMBER(6) PRIMARY KEY,  
  name VARCHAR2(25) NOT NULL,  
  salary NUMBER(8,2),  
  commission_pct NUMBER(2,2)  
);
```

2. Type the following:

```
INSERT INTO sales_reps(id, name, salary, commission_pct)  
  SELECT employee_id, last_name, salary, commission_pct  
  FROM   employees  
  WHERE  job_id LIKE '%REP%';
```

3. You should see:

```
4 row created.
```

UPDATE

- You can modify existing rows by using the UPDATE statement.

- SQL STATEMENT

UPDATE table

SET column = value [, column = value, ...]

[WHERE condition];

- In the syntax:
 - table is the name of the table
 - column is the name of the column in the table to populate
 - value is the corresponding value or subquery for the column
 - condition identifies the rows to be updated and is composed of column names, expressions, constants, subqueries, and comparison operators

UPDATE

- **NOTE:** In general, use the **primary key** to identify a single row.
- Using other columns can unexpectedly cause **several rows** to be updated.

TRY THIS

- Specific row or rows are modified if you specify the **WHERE** clause:

TRY THIS

1. Type the following:

```
UPDATE employees  
SET    department_id = 70  
WHERE  employee_id = 113;
```

2. You should see:

```
1 row updated.
```

- All rows in the table are modified if you omit the **WHERE** clause:

TRY THIS

1. Type the following:

```
UPDATE copy_emp  
SET    department_id = 110;
```

2. You should see:

```
22 row updated.
```

UPDATE

- You can update multiple columns in the SET clause of an UPDATE statement by writing multiple subqueries.

- **SQL STATEMENT**

UPDATE `table`

SET `column` = (SELECT `column`

FROM `table`

WHERE `condition`)

[,`column` =

(SELECT `column` FROM `table`

WHERE `condition`)]

[WHERE `condition`];

- **NOTE:** If no rows are updated, the message “0 rows updated” is returned.

TRY THIS

- In the next example, you will **update** employee 114's **job** and **salary** to **match** that of employee 205.

TRY THIS

1. Type the following:

```
UPDATE employees
SET    job_id = (SELECT job_id
                  FROM    employees
                  WHERE    employee_id = 205),
       salary = (SELECT salary
                  FROM    employees
                  WHERE    employee_id = 205)
WHERE  employee_id = 114;
```

2. You should see:

```
1 row updated.
```

Violating Constraints

```
UPDATE employees
SET    department_id = 55
WHERE  department_id = 110;
```

```
UPDATE employees
      *
ERROR at line 1:
ORA-02291: integrity constraint (HR.EMP_DEPT_FK)
violated - parent key not found
```

Department 55 does not exist.

DELETE

- You can **remove** existing rows from a table by using the **DELETE** statement:

- **SQL STATEMENT**

DELETE [FROM] **table**

[WHERE **condition**];

- **In the syntax:**

- **table** is the table name.
- **condition** identifies the rows to be deleted and is composed of column names, expressions, constants, subqueries, and comparison operators.

- **NOTE:** If no rows are deleted, the message “0 rows deleted” is returned.

TRY THIS

- Specific rows are deleted if you **specify** the **WHERE** clause:

TRY THIS

1. Type the following:

```
DELETE FROM departments  
WHERE department_name = 'Finance';
```

2. You should see:

```
1 row deleted.
```

- All rows in the table are deleted if you **omit** the **WHERE** clause:

TRY THIS

1. Type the following:

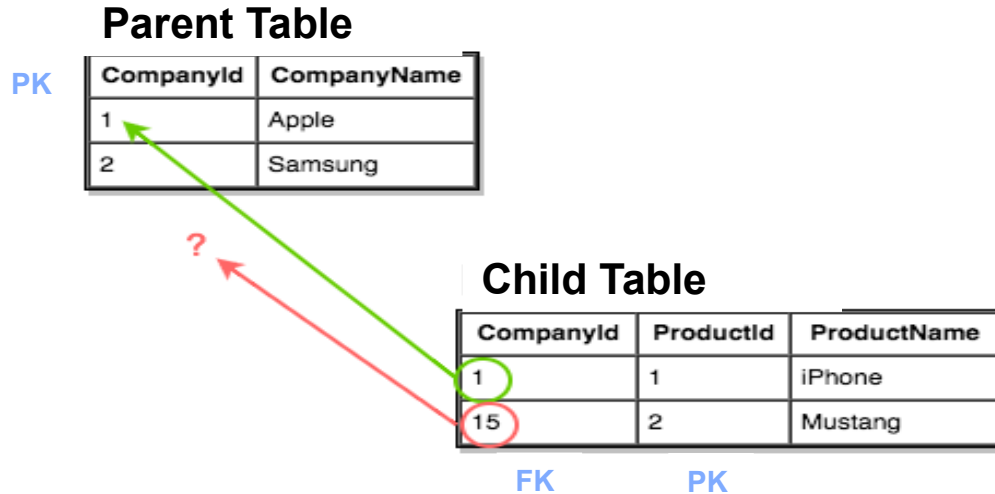
```
DELETE FROM job_grades;
```

2. You should see:

```
6 rows deleted.
```

Violating Constraints

- You **cannot delete** a row that contains a **primary key** that is used as a **foreign key** in another table.



TRY THIS

TRY THIS

1. Type the following:

```
DELETE FROM departments
WHERE      department_id = 60;
```

2. You should see:

```
DELETE FROM departments
          *
ERROR at line 1:
ORA-02292: integrity constraint (HR.EMP_DEPT_FK) violated -
child record found
```

3. If you try the following, does it execute successfully? Why?

```
DELETE FROM departments
WHERE      department_id = 190;
```